

Guide to Engineering Macroeconomic Features for 8-Hour Interval Models

Machine learning models can benefit greatly from macroeconomic context, but integrating such data at an 8-hour resolution poses unique challenges. This guide covers:

- **Key macroeconomic and market-based indicators (U.S. and global)** that can be adapted to 8-hour features.
- **Feature engineering techniques** (returns, spreads, momentum, volatility, rate of change, z-scores, PCA, etc.).
- **Data sources and APIs** (free options for Python, Rust, C++ with real-time and historical access).
- **Adapting low-frequency indicators** (e.g. CPI, PMI) to 8-hour frequency via nowcasting or market proxies.
- **Managing mixed frequencies** (feature update schedules, data alignment, and merging strategies).

Throughout, we include tables of recommended indicators, examples of feature transformations, and lists of data providers with notes on API access and rate limits.

1. Key Macroeconomic & Market Indicators for 8-Hour Features

Overview: We want indicators that provide broad economic and market context. Some are *market-based and high-frequency* (updating intraday or daily), while others are *official macro indicators* released at lower frequency (weekly, monthly, quarterly). Both U.S. and global indicators are considered. Crucially, focus on forward-looking measures that can influence market expectations ¹. The table below summarizes recommended indicators, their usual frequency, and how they can be adapted to an 8-hour model input.

Table 1 – Recommended Indicators and 8-Hour Adaptation

Indicator (U.S. unless noted)	Typical Frequency	Adaptation for 8-Hour Features
Equity Index (e.g. S&P 500) – broad market sentiment, growth proxy	Trades intraday on market hours (U.S. daily ~6.5h session; futures trade nearly 24h)	Use intraday prices or futures to get 8h interval values. Compute 8h returns (percent change) for momentum. Use rolling 24h returns (covering 3×8h periods) to capture daily trend.
Global Equity Indices (e.g. Euro Stoxx 50, Nikkei 225, MSCI World) – global risk appetite	Most trade in local market hours (daily); some ETFs trade nearly 24h	Include major foreign index levels or returns to reflect overseas market moves. Can use last available price and hold constant during their off-hours, or use related futures if available for continuous updates.

Indicator (U.S. unless noted)	Typical Frequency	Adaptation for 8-Hour Features
Treasury Yields (10-year, 2-year) – interest rates & economic outlook	Daily official quotes (e.g. from U.S. Treasury or FRED). Intraday available via bond futures or real-time feeds	Use latest daily yield and carry forward during the day ² . For intraday sensitivity, use futures or infer yield changes from bond ETFs. Also consider yield curve spread (e.g. 10y minus 2y yield) as a feature (update daily).
Corporate Bond Spread (e.g. High Yield OAS, BAA-AAA spread) – credit risk indicator	Daily (indices from FRED or ICE/BofA). No intraday trades, but respond to market conditions daily	Use daily value as a feature (forward-fill through 8h intervals). This reflects credit risk appetite. E.g. BofA High-Yield Index Option-Adjusted Spread (daily %).
Volatility Index (VIX) – equity volatility/risk aversion	Intraday (CBOE trades VIX during U.S. hours); widely quoted daily close	Use daily closing VIX as a feature to represent market fear. Forward-fill for intraday intervals. (If finer detail needed overnight, use VIX futures or realize vol from index moves.)
Crude Oil Price (WTI or Brent) – commodity, inflation and growth proxy	Trades nearly 24h in futures markets (continuous)	Use the latest price each 8h. Compute 8h returns or 1-day % change. Oil captures global demand/supply sentiment and can be interpolated across 8h windows from futures data.
Gold Price – safe-haven asset, inflation hedge	Trades nearly 24h (spot and futures)	Similar to oil: use current price/return each interval. Gold indicates risk aversion and inflation expectations.
FX Rates (e.g. EUR/USD, USD/JPY) or Dollar Index (DXY) – currency market sentiment, global capital flows	24h continuous trading (Forex)	Use 8h returns or changes. FX reflects relative economic outlooks and risk sentiment around the clock. The Dollar Index (DXY) (a basket) is a good single proxy – it often moves with global risk (USD up in risk-off).
Cryptocurrency Price (e.g. BTC) – alternative risk asset, 24/7 market	24/7 continuous trading (no breaks)	(Optional) Bitcoin or others can indicate global liquidity and risk sentiment off-market hours. Use 8h returns/volatility.
PMI (Purchasing Managers Index) – business activity (e.g. ISM Manufacturing PMI)	Monthly release (e.g. first of month for prior month)	Low-frequency official data. Treat as constant for the month, or use a nowcast approach (see Section 4). Could create an 8h feature that jumps on release (carry last value until update). Augment with high-freq proxies like industrial metal prices (e.g. Copper) to infer mid-month trends.

Indicator (U.S. unless noted)	Typical Frequency	Adaptation for 8-Hour Features
Inflation (CPI or PCE price index) – consumer price inflation rate	Monthly release (CPI mid-month for prior month)	Use last official value and forward-fill daily. For more dynamism, use inflation breakevens (market-implied inflation from TIPS vs Treasuries) which update daily as a proxy for inflation expectations. Additionally, use nowcasting (Section 4) for a daily updated CPI estimate ³ .
GDP Growth (quarterly % change) – broad economy	Quarterly official release (BEA advance estimate)	Between releases, use a running GDP nowcast that updates frequently ⁴ . E.g. the Atlanta Fed's GDPNow updates the GDP forecast throughout the quarter as new data arrives ⁴ . You can incorporate the latest GDPNow estimate as a feature (updated whenever a relevant report comes out).
Employment (e.g. U.S. Nonfarm Payrolls, Unemployment Rate) – labor market health	Monthly (usually first Friday for NFP, prior month)	Use last value, forward-filled. To get higher-frequency insight: include Weekly Initial Jobless Claims (released every Thursday) as a weekly feature that can be aligned to the nearest 8h interval of release. Also track market-based proxies like consumer confidence or job openings if available more frequently.
Consumer Confidence / Sentiment (e.g. U of Michigan CSI)	Monthly or bi-monthly (Michigan prelim and final)	Low-frequency – forward-fill last value. As proxy, monitor stock retail sectors or surveys that might move with confidence. Alternatively, use social media sentiment or Google Trends (if available) as high-freq sentiment indicators.
Global Macro Indicators – e.g. Eurozone CPI, China PMI, etc.	Mostly monthly or quarterly official data by country/region	If global exposure is needed, include major foreign macro indicators similarly (last known value + nowcasts or proxies). For example, China PMI monthly (use commodity prices as proxy intramonth), German Ifo or ZEW monthly sentiment (use Euro Stoxx or EUR exchange rate as proxy intramonth). These can provide international context but require same adaptation techniques (carry last value, update on release).

Why these indicators? They cover **financial markets (stocks, bonds, FX, commodities)** which are *leading or coincident signals* for economic conditions ¹, as well as **official economic metrics** (growth, inflation, sentiment) which directly tie to economic fundamentals but need interpolation. Market-based measures (prices, yields) are inherently higher frequency and often **discount future expectations** in real-time ¹. For example, major market indexes, bond yields/curve, and commodity prices “**provide meaningful statistics about the direction of the economy**” and reflect forward-looking expectations ¹. These can

be sampled or aggregated to 8-hour intervals to inject up-to-date context into your model. Official indicators, while critical, must be transformed to fit an 8h timeframe (we address this in Section 4).

Global vs U.S.: If your prediction target is influenced by global factors (e.g. a globally traded asset), incorporate *global indicators* too. For instance, the U.S. 10Y yield is key for U.S. markets, but Germany's 10Y Bund yield or Japan's Nikkei index might add information about global conditions. Many such series are available via FRED or other sources (FRED alone hosts **hundreds of thousands of international time series** ⁵). Balance the feature set to include major economies or markets relevant to your domain.

2. Feature Engineering Techniques for Macroeconomic Indicators

Raw values of indicators often need transformation to be most useful in a predictive model. By engineering features – e.g. computing returns, spreads, or normalized changes – we can capture the **dynamic behavior** of macro indicators and make them comparable. Below are common feature engineering techniques and examples of how to apply them to the indicators:

Table 2 – Common Feature Transformations

Feature Transformation	Definition / Example	Purpose
Return (Percent Change)	Compute relative change over a period. e.g. 8h log-return = $\log(\text{price}_t / \text{price}_{t-1})$. For a monthly series, use percent change from last month's value (when updated).	Puts data on a consistent percentage scale, captures momentum and direction. Using returns (or diffs) also helps stationarize non-stationary series (like prices or index levels).
Spreads & Yield Curves	Difference between two related metrics. e.g. yield spread = 10y Treasury yield – 2y yield; credit spread = corporate bond yield – risk-free yield; FX carry spread = interest rate diff between two currencies.	Spreads often indicate relative value or risk premium (e.g. widening credit spread implies risk-off). The 10y–2y yield curve spread is a famous recession predictor. Such features highlight changing relationships between instruments.
Momentum (Rolling Change)	Cumulative return or change over a longer window. e.g. 3-period momentum = $\text{price}_t / \text{price}_{t-3*8h} - 1$ (one-day change if 3 intervals = 24h); 7-day momentum for daily series = $\text{value}_{\text{today}} / \text{value}_{\text{last_week}} - 1$.	Captures trend and recent performance. Momentum features indicate whether an indicator has been rising or falling consistently. This can reflect sentiment (e.g. equity 1-week momentum, or inflation's acceleration vs deceleration).

Feature Transformation	Definition / Example	Purpose
Volatility (Rolling Std Dev)	Statistical volatility over a window. e.g. 3-day rolling standard deviation of 8h returns for S&P500; or 1-month rolling volatility of oil daily returns. Can also use realized volatility or implied vol (like VIX as a feature directly).	Measures uncertainty or risk. Higher volatility in an indicator (market turmoil or macro uncertainty) can foreshadow unstable conditions. Volatility features help the model account for regime changes (calm vs turbulent periods).
Rate of Change / Oscillators	Similar to momentum, often a normalized change. e.g. 14-period RSI (relative strength index) for an index; or simply percentage change over the last N intervals expressed as an annualized rate.	Highlights the speed of movement. For macro series, one might use month-over-month % changes (e.g. CPI monthly change) as the feature (which is essentially a rate of change). Oscillators like RSI can indicate overbought/oversold conditions in market prices.
Z-score (Standard Score)	Normalized value: $z_t = \frac{x_t - \mu_{window}}{\sigma_{window}}$, where μ and σ are mean and std over a rolling window or historical baseline. E.g. z-score of today's PMI relative to past 12 months, or z-score of current S&P level vs its 20-day moving average.	Normalization puts different indicators on a comparable scale and highlights extreme deviations. A z-score feature tells how unusual the current value is – e.g. a very low z-score for consumer confidence indicates exceptionally weak sentiment relative to recent history.
Principal Components (PCA factors)	Linear combination of multiple related features into one or few composite indices using PCA. E.g. take a set of 10 macro indicators (inflation, PMI, yields, etc.) and derive the first principal component as a “global factor”. ⁶	Reduces dimensionality and noise by condensing information. PCA-derived features are uncorrelated and often more stable. They can “ function better as features for predictive regressions, stabilizing coefficients and reducing noise ” ⁶ . For example, you might extract a “growth factor” from several economic series to use as a single feature.

Other ideas: You can combine basic transformations for more complex features. For instance, **rolling averages** (trends) vs current value (to detect mean reversion or divergence), **max/min over period** (to capture range or extremes), **cross-features** (like product or ratio of two indicators), or even **text-based features** (if using news sentiment as macro input). Technical analysis indicators (moving averages, RSI, MACD, etc.) applied to price series are essentially specialized forms of momentum or volatility features – these can be included if relevant.

When engineering features, be mindful of **timing and lookahead bias**. All transformations should use **past and up-to-time-t data only** when computing features for time t. For example, if you compute a 7-day rolling average for an 8h model, ensure at time t it only uses data up through t (no future leakage).

Feature scaling: Some algorithms benefit from scaled inputs. Using returns, percentage changes, or z-scores inherently provides scaling. If you use raw levels (e.g. an interest rate of 5% vs an index level of 4000), you may want to scale or standardize these so that one feature doesn't unduly dominate due to units. Z-scores or min-max scaling can help put features on comparable scales.

Example – S&P 500 feature set: Suppose you include the S&P 500 index. Rather than the raw price, you could feed: (a) the 8-hour log-return of S&P, (b) the **24-hour return** (span of 3 intervals) for a slightly longer trend, (c) a **5-day momentum** (% change over last 15 intervals), (d) a **volatility measure** such as std. deviation of S&P 8h returns over the last 3 days (9-interval window), (e) perhaps the **z-score of S&P's price vs its 1-month average**. This gives a richer representation of the equity market state (recent trend, volatility, and relative level). Similarly, for a yield curve you might include the 10y-2y spread itself as a feature, plus the **rate of change of that spread** over the past month to indicate if the curve is steepening or flattening quickly.

Example – CPI inflation feature: The CPI is monthly, so one feature could be **year-over-year CPI %** (updated monthly). But an 8h model would see that value static for ~30 days, then a jump. You might instead use the **monthly change (0.x% m/m)** as the feature (which is essentially the "inflation momentum"). Additionally, you could include an **inflation nowcast (daily)** that updates subtly day by day (see Section 4), or **market-based inflation expectation** (like the 5-year Breakeven Inflation rate from TIPS, which is daily from markets) as a continuously moving inflation proxy. These transformations make a low-frequency series more model-friendly.

3. Data Sources and APIs for Macro & Market Data (Python, Rust, C++)

Collecting the data for these indicators requires reliable sources that offer both historical data **and** real-time updates. Below we list popular free (or freemium) data providers, the type of data they offer, and how to access them in Python, Rust, or C++. We focus on providers mentioned (FRED, Alpha Vantage, Tiingo, Yahoo Finance, Quandl, Binance) and a few others, noting API availability and usage limits:

- **FRED (Federal Reserve Economic Data) – Huge repository of macroeconomic time series** from the St. Louis Fed. Over 672,000 series covering U.S. and international data ⁵ (interest rates, GDP, CPI, sentiment indexes, financial indexes like VIX, etc.). Historical data is well-maintained. **API:** Yes (free with API key). Generous rate limits (~120 requests/minute) ⁷. In Python, use the `fredapi` library or `pandas_datareader` (e.g. `DataReader('GDP', 'fred')`). In Rust, use crates like `fred_rs` ⁵ or `fred_api`. In C++, no official library – use HTTP GET (e.g. with libcurl) to FRED's REST endpoints (which return JSON or XML). FRED is excellent for **official indicators (GDP, CPI, etc.)** and some daily market proxies (e.g. FRED has daily Treasury yields, corporate spreads, financial stress indices). **Real-time:** FRED data updates shortly after releases (not tick-by-tick, but you can pull the latest value as soon as it's published). Rate limit ~120/min, which is effectively not restrictive for most use cases ⁷.
- **Alpha Vantage – Market data API** (stocks, ETFs, forex, crypto, technical indicators, and some macro series under "economic indicators"). Free tier requires an API key (just sign up). **Usage limits:** *Free: 5 API calls per minute and 25 calls per day* ⁸ (they significantly lowered the daily limit, used to be 500/day) – so you must economize calls or cache data. Data provided includes daily and intraday time

series for equities and FX, and endpoints for things like GDP, CPI, Treasury yields, etc. ⁹ (those macro endpoints likely mirror FRED or other sources). **Python:** use the official `alpha_vantage` Python package or any HTTP client (`requests`). **Rust:** no official crate, but easy to call REST API with `request` or similar; community crates like `alpha_vantage` may exist. **C++:** use libcurl or C++ REST SDK to hit their JSON endpoints. Note the strict free limits – you may need to batch your data downloads (and sleep between calls) or upgrade if 25/day is insufficient ¹⁰. For real-time usage, Alpha Vantage offers **intraday stock and FX data** (down to 1-min bars), but again limited by call frequency – e.g. fetching a full intraday series counts as one call (which might return hundreds of points). If you need frequent intraday updates, other sources or a paid plan would be necessary.

- **Yahoo Finance** (unofficial API) – Yahoo provides a wide range of **financial market data**: stock prices, indexes, FX, commodities, etc., often for free via their website. The official API was discontinued in 2017 ¹¹, but you can access data through *unofficial methods*. **Python:** the popular `yfinance` library scrapes Yahoo's JSON endpoints behind the scenes. It's free and requires no API key. Yahoo's data includes daily history for decades and intraday data (to some extent). **Intraday detail:** yfinance can fetch high-frequency data (e.g. 1-min bars) but only for the last ~7 days (Yahoo limits recent intraday history) and up to 60 days for <=1h intervals ¹². So for 8-hour bars, Yahoo might give ~60 days of history if using their "interval=8h" (which is not standard, but 1h or 4h can be aggregated). For longer history of intraday data, you'd need another source. **Rust:** there are community crates (e.g. `yahoo_finance_api` or `yfinance-rs`) that wrap Yahoo's data endpoints ¹³. **C++:** no official SDK, but one can download CSV data from Yahoo's web endpoints (requires handling a cookie and crumb token) or use a C++ HTTP client to query the same endpoints that yfinance uses. Yahoo has no set rate limit (since it's web scraping, heavy use might get your IP temporarily blocked), but moderate use is fine. **Data:** Yahoo covers equities worldwide, indices (use their caret ticker like `^GSPC` for S&P 500, `^VIX` for VIX, `^TNX` for 10Y yield), futures (limited), FX pairs, and crypto. It's a great free source for many *market-based indicators* we listed, especially for daily or end-of-day data. For real-time streaming, Yahoo is not ideal (consider broker APIs or paid feeds if needed).

- **Tiingo – Financial data API** offering end-of-day stock prices, crypto, forex, and news. Basic use is free with registration (includes *IEX stock prices* and some data, with limits). Tiingo does not impose minute-by-minute rate limits for free users, but has a **daily/monthly cap** (undisclosed; anecdotal evidence suggests on the order of a few hundred symbols per day for free). If you need more, they have a paid plan (\$10–30/month range for individuals). **API:** Yes (key required). **Python:** `tiingo` Python client or use `requests` with the Tiingo REST endpoints (JSON). **Rust/C++:** no dedicated crate, but simple to use REST calls with an API key in header. Tiingo's free data is good for **daily historical** prices (U.S. equities, some international, and crypto) and also has a news API. Notably, Tiingo offers real-time **IEX** feed for U.S. stocks (with a delay for free accounts). They explicitly state no short-term rate limit ("we do not rate limit to minute or second" for free, but have an **hourly allocation** that free accounts shouldn't exceed) ¹⁴ ¹⁵. So for moderate requests it's fine, but downloading thousands of tickers will hit a limit. Use Tiingo as a reliable alternative to Yahoo for daily data (with less chance of web-scraping issues).

- **Quandl (Nasdaq Data Link)** – an aggregator of various datasets, including many macroeconomic databases and some financial market data. **API:** Yes (free account gives an API key). Many **free datasets** (e.g. FRED data is mirrored on Quandl/Nasdaq Data Link, as are World Bank data, etc.). Also some premium (paid) datasets. **Python:** use `nasdaqdatalink` (formerly `quandl`) Python

package. **Rust/C++:** no official library, but simple REST calls (CSV or JSON). Quandl can be handy for one-stop access to certain data like commodity prices, global macro series, etc. For example, they host historical data for OECD leading indicators, some global stock indexes, etc. If you use Quandl's free tier, note the call limit (Nasdaq API is around 50 calls per day for free users, similar to old Quandl limit). However, each call can retrieve an entire timeseries, so it's often enough.

- **Crypto Exchange APIs (e.g. Binance)** – useful for **real-time, high-frequency data** and 24/7 coverage. Even if you're not directly predicting crypto, Bitcoin or other crypto prices can serve as a proxy for global liquidity or risk sentiment in off-hours. Binance is a top exchange with a robust API. **API:** Yes, free. **REST endpoints** for historical and latest data, and **WebSocket** streams for real-time ticks/order books. **Limits:** generous (e.g. Binance REST has weight limits like 1200 requests/minute on certain endpoints, but a single kline query might count as 1). For an 8h interval model, you might only need to ping a few endpoints or open a stream. **Python:** use `python-binance` library or `ccxt` (which supports multiple exchanges including Binance). **Rust:** use community crates (e.g. `binance` crate ¹⁶ or `binance-async`). **C++:** Binance provides an official C++ library on GitHub (binance-exchange/binacpp) ¹⁷, and there are third-party implementations (e.g. using Boost.Beast for websockets). Binance data can provide *continuous price series* for crypto assets which trade 24/7 – one could use BTC/USD as an input feature that updates every 8h (or even as a proxy for weekend market sentiment when other markets are closed). Similarly, other exchanges or CoinMarketCap/CoinGecko APIs could be used for crypto data if preferred (free, with rate limits).

- **Other useful free sources:**

- **IEX Cloud** (offers U.S. stock prices, some macro like yield curve points, with a free tier of 50k messages per month; has Python/Rust/C++ clients but requires account setup).
- **ECB SDW (Statistical Data Warehouse)** for European macro data (has an API, less user-friendly JSON).
- **World Bank API** for global economic indicators (free JSON API, but annual data mostly).
- **OECD API** for economic indicators (free).
- **TradingEconomics API** (has a free tier for some indicators with registration).
- **St. Louis Fed's ALFRED** for historical revisions (if you need vintage data).
- **Yahoo Finance (RapidAPI)** – Yahoo has an official paid API via RapidAPI if scraping is undesirable.
- **Polygon, Finnhub, Alpaca, TwelveData, etc.** – various freemium market data APIs. Each has Python clients and different limits (often 500–1000/day free). These can provide intraday stock data or forex beyond what Alpha Vantage's limits allow.

When choosing sources, consider **update frequency**: For example, FRED will give you daily closing yields; if you need tick-by-tick intraday yields, you'd need a different source (maybe a broker or trading platform API). For most macro features, daily granularity is sufficient given their slow movement (except market prices). You might use a combination: e.g. FRED for history and daily updates of CPI, but Yahoo or Alpha Vantage for intraday moves of S&P or EUR/USD.

Also mind **data quality and consistency**: e.g. Yahoo data is free but sometimes has small discrepancies or occasional missing points vs official sources. FRED data is official quality for macro series. Plan to clean or backfill data as needed.

Rate Limit management: If using free APIs in production, implement checks or delays to avoid hitting limits (e.g. Alpha Vantage returns a specific message if limit exceeded ¹⁸). Libraries like `alpha_vantage` for Python often handle a basic sleep between calls. For FRED, the generous limit means you rarely need to worry, but if you did a bulk load of thousands of series, you might throttle to <120/min. In Rust or C++, implement simple rate-limiters or scheduling of calls. Caching data (locally or in a database) is important – fetch new data points incrementally rather than entire history every time.

4. Adapting Low-Frequency Macro Indicators to 8-Hour Resolution

Many key macroeconomic indicators (GDP, CPI, PMI, etc.) are released only **monthly or quarterly**, yet our model operates at 8-hour intervals. We need strategies to incorporate the information from these low-frequency series without misleading or starving the model. Here are several approaches:

- **Forward-Filling (Step Hold):** The simplest method is to treat the indicator as a step function that updates on release. For example, if CPI (year-over-year %) in June is 3.0% and July's release (mid-July) says 3.2%, you would feed 3.0% for all 8h intervals up until the release time, then 3.2% thereafter. Essentially, the feature is "last known value" at any time. This ensures at each 8h step the model knows the latest info. You'd typically also include a **feature indicating a release just occurred** (e.g. a binary flag or the size of change on that interval) to help the model handle the jump. Forward-filling is easy (e.g. after merging the monthly series with your 8h timestamps, use pandas forward-fill ²). The downside is the feature is **piecewise constant** and may not capture gradual changes between releases. The model might also learn that the exact moment of update is significant – which could be useful if the target is affected by surprise in that macro data.
- **Linear or Elastic Interpolation:** Instead of a step, you can interpolate values in between releases. For example, if PMI was 50 last month and 52 this month, you could linearly increase it over the intervening time steps. This assumes a steady change and **smooths the jumps**. It might be reasonable for some trending indicators but risky for others (macro data often jump at release because of new information rather than actual gradual move). One could also use a **hold-and-jump** hybrid: hold the old value until a certain time (perhaps near the next release), then jump – but that's essentially forward-fill.
- **Nowcasting Models:** **Nowcasting** refers to predicting the current value of an economic indicator before it's officially released, using higher-frequency data. You can integrate nowcasts to update your feature more frequently. For instance, the Cleveland Fed produces a **Daily Inflation Nowcast** for CPI and PCE inflation ³ – providing **daily estimates of monthly inflation** using high-frequency inputs (like daily commodity prices, weekly retail gasoline prices, etc.) ¹⁹. By using such a nowcast, your "inflation" feature can *drift slightly day by day* instead of staying flat. Similarly, the Atlanta Fed's **GDPNow** is updated multiple times a month (after key releases) to forecast the current quarter's GDP ⁴. Including a GDP nowcast feature means you'll have a number that changes possibly every few days as new data arrive (e.g. employment report, retail sales, etc., each update the GDP estimate). The model thus sees a smoother evolution of GDP expectation rather than a quarterly jump. You can obtain these nowcasts from their sources (Atlanta Fed posts updates on their site and via press releases; you may need to scrape or manually integrate them, or check if they're in FRED – indeed, FRED has a series for GDPNow updated in a time series ⁴). Another example: nowcasts for other countries (e.g. ECB and various central banks publish nowcasting research, but not always as

easily accessible data). If none available, you could **train your own nowcast models** using related daily/weekly data, but that's a non-trivial project.

- **Market-Based Proxies:** Financial markets often anticipate economic changes. You can use *market data as a proxy* for macro indicators between releases:
- **Inflation:** Use market **breakeven inflation rates** (difference between nominal and TIPS yields) as a continuously updated inflation expectation. E.g. 5-Year, 5-Year Forward Inflation Expectation rate (from FRED) or daily 10y breakeven. These trade daily and incorporate inflation outlook. Also, commodity prices (oil, agriculture, metals) can indicate inflation pressures – one might construct a composite of commodity price changes as an 8h feature to proxy for inflation trends.
- **GDP / Growth:** Use broad asset indices that correlate with growth. For instance, industrial commodities (copper often nicknamed *Dr. Copper* for its diagnostic power on the economy) – copper price rising could imply industrial demand. Stock indexes (especially cyclical sectors) can serve as a proxy for growth sentiment. There are also aggregate “growth indices” like the OECD Weekly Tracker or Bloomberg’s weekly GDP tracker, etc. The New York Fed publishes a **Weekly Economic Index (WEI)** that uses high-frequency data to infer current weekly GDP growth (available on FRED). Such series can be scaled to match GDP and update weekly – much finer than quarterly.
- **Employment:** Use **weekly jobless claims** (already mentioned) – this is a good proxy for labor market turning points on a weekly basis. Also, consider online job postings or staffing indexes (if available) as more continuous signals.
- **Consumer sentiment:** Between monthly survey releases, use things like daily **Google Trends** for certain keywords, or **market sentiment indicators** (e.g. put-call ratios, consumer discretionary stock performance vs staples) to gauge confidence.
- **Monetary policy expectations:** The Fed’s policy rate changes only at scheduled meetings ~6 weeks apart. But the expectations for those moves trade daily in futures/overnight index swaps. If short-term interest rates matter, use futures-implied rate (e.g. Fed Funds futures rate for next meeting, or 2-year yield as a proxy) which moves continuously as expectations shift. That way the model sees the policy outlook continuously, not just the last set rate.
- **Miscellaneous:** Financial conditions indexes (like Goldman’s FCI or Chicago Fed’s NFCI) are weekly – could incorporate those for broad stress. The **St. Louis Fed Financial Stress Index** is weekly (in FRED). These combine many factors and update more frequently than crises themselves.
- **Scaling low-frequency data for high-frequency changes:** Another trick: instead of raw value, feed **change or surprise** as the feature at the time of release. For example, for Nonfarm Payrolls (NFP) which comes monthly, you could have a feature that is *zero most of the time*, and at the 8h window when NFP comes out, set it to the **percent surprise** (actual minus forecast). This way the model learns the impact of unexpected macro shocks. But that requires having consensus forecasts data to compute surprise – which is advanced and requires data (e.g. from Bloomberg or surveys) not freely available easily. If you don’t have forecast data, you might still use the actual change (which if the model learns context, could infer surprise relative to trend). This approach treats macro releases like events.

In practice, you might mix approaches. A good pattern is: **use forward-fill for the base level**, plus **add an auxiliary feature for nowcast or proxy that wiggles in between**. For instance, have one feature “CPI_yoy_last” (forward-filled last official YoY CPI) and another “Daily_infl_nowcast” (which moves slightly

each day based on commodities and known data). At the moment of release, `CPI_yoy_last` jumps to new value; the nowcast likely was near that value just before release. The model can use the nowcast for interim adjustments and still see the official number when out. This reduces the shock and potentially improves modeling continuity.

If creating your own nowcasts: you'd use a regression or ML model on higher-frequency inputs to predict the monthly figure. For example, **nowcast CPI**: inputs might include daily gasoline prices, exchange rates (dollar strength affects import prices), surveys of price trends, etc. The model (even a simple linear model) could update the estimate as those inputs change. While detailed nowcasting is beyond scope here, note that **macro nowcasting is an active field** – e.g. researchers use machine learning to nowcast inflation weekly ²⁰ or daily. You can also incorporate *partial data*: e.g. if halfway through the month and you have some weekly data, update the monthly projection proportionally.

- **Resampling and Frequency Alignment:** If using pandas, you might use `.resample()` to up-sample a monthly series to 8H frequency with a fill method. For example:
`df['CPI'].resample('8H').ffill()` will create the step function. Or use `merge_asof()` to align release timestamps to nearest 8H index and then `ffill` ²¹ ².

Example – Adapting PMI: U.S. ISM PMI comes on the first business day of each month for the prior month. You could: - Forward-fill last PMI value through the month. - Use daily data like **S&P Global PMI (flash estimates mid-month)** or regional Fed surveys (Philly Fed, etc.) as interim signals. - Use related markets: e.g. if PMI is manufacturing, the performance of industrial metal prices or stock indices of industrial companies might correlate. Create an “industrial commodity index” feature that’s daily (even 8h if using intraday commodity prices), which will rise or fall with manufacturing demand. - On PMI release day, you might also include a binary feature “PMI_release_today” so model can treat that time specially (if needed).

Example – Adapting Unemployment: Unemployment rate is monthly, but weekly jobless claims give a peek. You include *Unemp_rate* forward-filled, and *WeeklyClaims* which is updated every Thursday 8:30am EST. The weekly series can be aligned to the nearest 8h interval. In production, when it’s Thursday morning and new claims come, your feature pipeline would update that feature. The unemployment rate feature stays at last value until the next NFP day when it jumps.

Caution: If you forward-fill a value for a long time, the model might learn it doesn’t change often and could discount it. That’s why combining with something that *changes* in the interim (proxies or nowcasts) is helpful. Always ensure that any “smoother” you use (nowcast or interpolation) only uses information **available up to that time** – avoid peeking at the future official release. Using a separate high-frequency proxy or an explicit nowcast model helps enforce that.

Finally, be aware of **release timing**: Many U.S. indicators release at 8:30 AM ET or 10:00 AM ET, which will fall into a specific 8h bucket depending on your alignment (e.g. if your intervals start at midnight UTC, 8:30am NY is 13:30 UTC, which might fall in the 8:00-16:00 UTC interval). Make sure your feature updates at the correct interval (the one that covers the release time). In pandas, you might timestamp the new value at the release datetime and then forward-fill, so that any model interval after that time sees the new value. Also consider time zones – it may be easiest to convert everything to UTC internally.

5. Managing Feature Frequency, Data Alignment & Updates in Production

Working with mixed-frequency data (some features update every few hours or real-time, others daily or monthly) requires careful alignment and data management:

Frequency and Update Schedules: Establish how often each feature should update: - **Intraday market features** (prices, yields, etc.): can update every 8h or more often. For example, you might pull new price data for equities and FX every hour or continuously via websocket, and aggregate to your 8h bins. - **Daily features** (many macro proxies or indicators): update once per day (e.g. at market close or specific time). In an 8h model, this means the feature value will be the same for all intervals within that day (unless you choose to interpolate intraday, which usually isn't done for daily data). - **Weekly or monthly features:** update on their release schedule (e.g. Thursdays 8:30 for claims, first of month for PMI, etc.). These will stay constant until the new release comes.

It's useful to maintain a **metadata table** of features with their native frequency and next expected update time, so your production system knows when to fetch new data. For example, for CPI you know the calendar (release monthly around mid-month); for FOMC rate decisions, know the dates.

Data Alignment and Merging: Use a time index to merge features. In Python/pandas, a recommended approach is using `pd.merge_asof` (as-of join) to align a slower series to the closest prior timestamp of a faster series ²¹. This effectively forward-fills by merge. For example, if you have an 8h timeline as your primary index, you can merge_asof the monthly CPI dataframe onto it (on time), direction='backward' to get the last known CPI for each interval. Then `.ffill()` to carry the last value forward ². Alternatively, you can upsample the data: e.g. `monthly_df.resample('8H').ffill()`. The result is the same – each 8h row gets the latest available macro value.

Be cautious with **end-of-period vs start-of-period conventions**. Forward-fill (backward merge) assumes that at 8h interval *after* release you want the new value. If your interval exactly lands on a release time, decide if that interval should contain old or new value (likely new if release happened at, say, 9:00 which is within that interval). You might have to handle exact timestamps carefully (e.g. if release at 08:30, and you had an interval starting 08:00, you may treat the feature as updated at 08:30 within that window – one could break intervals at release times if needed, but that complicates uniform frequency).

Handling missing or irregular timing: If a daily series doesn't publish on weekends (e.g. stock prices), and your model still runs on weekends, you'd forward-fill Friday's value for weekend intervals. E.g. an 8h model running continuously might carry Friday's S&P 500 last price through Saturday/Sunday intervals (or you might prefer to mark them as NaN/unchanged). Decide according to your use case.

Time zone normalization: Align all times to a common zone (UTC is easiest) internally. Market data from Yahoo might be in New York time; adjust those timestamps. FRED data often has date with no time (meaning end-of-day). You might assume end-of-day as 00:00 UTC of the next day or something – just be consistent so merges work.

Feature Store / Database: In a production setting, consider storing your features in a time-series database or a feature store. For example, you might have a database table with columns [timestamp, S&P_return,

Oil_return, 10y_yield, CPI_yoy, ...]. Each time an 8h interval passes (or when new data arrives), you update this store. Tools like **Feast** (feature store) can manage offline vs online features and their update frequencies, ensuring your model serving gets the latest features with correct timestamps. Even without a formal feature store, a simple script can update a CSV or database periodically for these features.

Synchronization: Since data comes from multiple sources, ensure your fetching jobs run in the correct order and frequency. E.g. you might run a job to fetch daily closes at 5pm each day for some features, whereas a job for intraday data runs every 8 hours. Using an orchestrator (like Apache Airflow or cron jobs with dependencies) can automate this.

Real-time streaming updates: For certain features, such as live market prices or crypto, you could maintain a streaming listener (WebSocket) and update an in-memory value. If your model inference is event-driven (say you predict every 8h on the dot), you can sample the latest value from the stream at that time. Languages like Python can use asyncio or threaded approaches with websockets; Rust has async runtime (Tokio) good for this; C++ can use Boost.Beast for websockets. For example, subscribe to Binance's BTCUSDT ticker stream and keep updating a global variable, so at inference the last price is readily available.

Merging in Rust/C++: Without pandas, you might manually implement forward-fill merges. One approach is to load all series sorted by time, then iterate through the main timeline and look up the latest index from the macro series. Or use a library: e.g. Rust's **Polars** DataFrame supports join_asof (similar to pandas), which could be very handy to align time series in Rust. C++ doesn't have native DataFrame, but one can sort vectors of structs by time and do binary search to find the previous data point for a given time. If data volume is huge and latency is critical, consider a specialized time-series database (like kdb+/q, although not Python/Rust/C++ friendly out-of-the-box, or InfluxDB etc.) to query the last known value as of a time.

Feature Update Frequency: Not every feature needs to be recomputed every 8 hours if its data hasn't changed. Design your pipeline to only update when needed: - E.g., your code can check if a new value is available (via API or web) for a macro feature before updating. Many APIs provide a "last updated" timestamp or you could keep track. - For daily series, an update check more than once a day is redundant (except maybe to catch late postings). - For intraday series, decide if you want to update exactly on the 8h schedule or continuously and then sample. If your 8h intervals are say 0:00, 8:00, 16:00 UTC, you might query APIs a few minutes after those times to get closing values or last prices for that interval.

Data Quality and Backfilling: Ensure you backfill historical data for all features to train your model. Merge the history with proper alignment (pandas join or custom) and handle any missing periods. If an indicator started later than others (common with new economic series), you might have NaNs for earlier dates – decide whether to fill with something (e.g. neutral values or drop those periods from training).

Mixed Frequency Modeling Considerations: In academic literature, **mixed-frequency models** (like MIDAS regression) explicitly handle different frequencies by weighting lagged higher-frequency features to predict low-frequency targets. In our case, we are predicting at a high frequency with some low-freq inputs – effectively we've solved it by filling in values. It's usually acceptable as long as we do it in a way that the model doesn't "see the future." For example, forward-filling means the model has no knowledge of the next release until it happens (which is correct). Interpolation might inadvertently leak a bit of future trend (if one assumes a straight line towards the future value – but since we don't know that future value during training either, it's not leaking actual future, just a conjecture).

Example Implementation (Python): Suppose we have dataframes for S&P 500 8h prices and monthly CPI. We can do:

```
# Ensure dataframes have datetime index
sp.index = pd.to_datetime(sp.index)
cpi.index = pd.to_datetime(cpi.index)
sp = sp.sort_index()
cpi = cpi.sort_index()
# resample CPI to 8H frequency, forward-fill values
cpi_8h = cpi['CPI_yoy'].resample('8H').ffill()
# align to SP500 index (assuming SP has every 8h timestamp)
cpi_aligned = cpi_8h.reindex(sp.index, method='ffill')
# Now cpi_aligned at each sp 8h timestamp is last CPI
```

Or using `merge_asof`:

```
merged = pd.merge_asof(sp.reset_index(), cpi.reset_index(), left_on='index',
right_on='index', direction='backward')
```

This would attach the latest CPI to each S&P timestamp.

Production Tip: After assembling all features per 8h interval, you'll likely have a row with perhaps a timestamp and 20+ feature columns. It's good to store this as a CSV or Parquet file for training. In production, to generate the latest feature vector, you would gather the latest values (some from caches or DB, some from APIs) and ensure they align to "now". If using Python in production, libraries like **schedule/ APScheduler** can be used to run tasks (e.g. fetch daily data at 5pm, fetch intraday data every 8h, etc.). In Rust/C++, you might have a long-running service sleeping or using OS cron to trigger fetches.

Ensuring data consistency: All features should refer to the same effective time. If one feature lags (e.g. due to API delay), you might temporarily have an older value. This can introduce jitter. A robust solution is to timestamp each feature's update and, if a critical one is missing, maybe delay prediction until it's in (depending on use case). Alternatively, design the model to tolerate slightly stale inputs for some features (it might anyway, if e.g. CPI is one week old, it's not end of world).

Finally, **test your pipeline** thoroughly: simulate through a few known macro release dates to ensure values transition as expected in the 8h sequence. It can be helpful to log or plot the feature values over time to verify that forward-fills and jumps occur correctly.

By engineering a rich set of macroeconomic and market features and managing their different frequencies, you provide your model with valuable context about the economic regime and market sentiment for each 8-hour period. The combination of **carefully chosen indicators**, appropriate **feature transformations**, and a robust data pipeline (leveraging free APIs and proper alignment) will help improve your model's accuracy and robustness in both historical backtests and live production. With the above guide, you can confidently

incorporate everything from daily oil price swings to monthly inflation data into your 8h predictive modeling. Good luck!

Sources:

- Investopedia – Key economic indicators and market measures ¹
- St. Louis Fed (FRED) – FRED API and data availability ⁵ ; FRED rate limit ~120 requests/min ⁷
- Macroption – Alpha Vantage free API limits (5/min and 25/day) ⁸
- Yahoo Finance via yfinance – intraday data limitations (up to 60 days) ¹²
- Macrosynergy Research – Benefits of PCA for macro features ⁶
- Cleveland Fed – Daily inflation nowcasts (provides daily CPI/PCE estimates) ³
- Atlanta Fed – GDPNow updates throughout quarter with new data (nowcasting GDP) ⁴
- Pandas documentation – merge_asof for time alignment ²¹ and forward-fill usage ²
- Alpha Vantage API docs – Economic indicators endpoint (GDP, CPI, etc.) ⁹ and Premium policy ¹⁰ .

¹ Economic Indicators That Help Predict Market Trends

<https://www.investopedia.com/articles/economics/08/leading-economic-indicators.asp>

² ²¹ Merging Ordered and Time-Series Data in Python with pandas | by Damilare Daramola | Medium

[https://iamdamilare13.medium.com/merging-ordered-and-time-series-data-in-python-with-pandas-7692733ab1fd?](https://iamdamilare13.medium.com/merging-ordered-and-time-series-data-in-python-with-pandas-7692733ab1fd?source=rss-----data-5)
source=rss-----data-5

³ Inflation Nowcasting

<https://www.clevelandfed.org/indicators-and-data/inflation-nowcasting>

⁴ GDPNow (GDPNOW) | FRED | St. Louis Fed

<https://fred.stlouisfed.org/series/GDPNOW>

⁵ fred_rs - Rust

https://docs.rs/fred-rs/latest/fred_rs/

⁶ Using principal components to construct macro trading signals | Macrosynergy

<https://macrosynergy.com/research/using-principal-components-to-construct-macro-trading-signals/>

⁷ NEWS

<https://cran.r-project.org/web/packages/fredr/news/news.html>

⁸ ¹⁰ ¹⁸ Alpha Vantage API Request Limits - Macroption

<https://www.macroption.com/alpha-vantage-api-limits/>

⁹ API Documentation | Alpha Vantage

<https://www.alphavantage.co/documentation/>

¹¹ ¹² yfinance Library - A Complete Guide - AlgoTrading101 Blog

<https://algotrading101.com/learn/yfinance-guide/>

¹³ yahoo_finance - Rust - Docs.rs

<https://docs.rs/yahoo-finance>

¹⁴ ¹⁵ API Limit for Free User : r/Tiingo

https://www.reddit.com/r/Tiingo/comments/1bqpd6k/api_limit_for_free_user/

16 binance - crates.io: Rust Package Registry

<https://crates.io/crates/binance>

17 binance-exchange/binacpp: Binance C++ library - GitHub

<https://github.com/binance-exchange/binacpp>

19 US - Cleveland Fed Inflation Nowcast: CPI | US Prices | Collection

<https://en.macromicro.me/collections/5/us-price-relative/27688/us-cleveland-inflation-cpi>

20 Now-casting inflation using high frequency data - ScienceDirect

<https://www.sciencedirect.com/science/article/abs/pii/S0169207013000046>